

Formal Method Specification — RCA with Knowledge Graph and Chroma

Updated: April 6th, 2026

Baseline: Orchestrator v3.2, Schema set v3.2, validated fixture-driven workflow

- The method is intended to be machine-readable, auditable, and explainable for nuclear equipment reliability investigations
- Knowledge-graph structure (Neo4j) and vector retrieval (Chroma) are as of now both inputs to the RCA pipeline
- LLM output is advisory. The analyst remains the final decision-maker
- The deterministic fallback synthesis path is currently the production path. The LLM path (OllamaLLMClient) exists but is not yet validated end-to-end.

1. Purpose and Scope

This method defines the current RCA workflow for single-event analysis augmented by historical events, document corpora, system architecture, telemetry, and maintenance/governance data. The target user is a system engineer or reliability engineer performing structured troubleshooting with analyst sign-off.

The workflow is explicitly decision support, not decision automation. All final technical and regulatory conclusions remain the responsibility of the licensed engineer. The system does not self-approve a conclusion.

2. End-to-End Artifact Chain

Stage	Primary Artifact	Current Role in Workflow
Run setup	run_context	Establishes run identity, configuration, and input validation result
Event framing	event.json	Defines event identity, severity, event type, timing, and symptom signature
Telemetry snapshot	telemetry_summary.json	Summarizes abnormal signals, timing, and supporting statistics
KG narrowing	kg_context.json	Defines candidate components, paths, failure modes, documents, and historical context
Temporal reasoning	tskr_patterns.json	Captures temporal relations, lag statistics, latency alignment, contradiction flags, and matching signals
Candidate generation (pre-evidence)	causality_candidates_v1.json	Ranks competing hypotheses using structural, temporal, telemetry, evidence-prior, and governance dimensions. Includes recurrence and common-cause enrichment.
Evidence retrieval	evidence_bundle.json	Holds ranked snippets retrieved from Chroma with per-candidate support/contradiction/context classification
Candidate refinement (post-evidence)	causality_candidates_v2.json	Re-scores candidates using retrieved evidence. Updates evidence posture, composite score, and threshold pass/fail.
Ishikawa evaluation	ishikawa_matrix.json	Organizes findings into fishbone categories for structured RCA review

Stage	Primary Artifact	Current Role in Workflow
Synthesis	rca_card.json	Produces the analyst-facing hypothesis summary, alternatives, evidence, actions, and review questions
Validation	validation reports	Per-artifact and cross-artifact consistency checks
Review and commit	run_manifest + analyst decision	Controls writeback readiness and final KG update

Important architectural note: The candidate set goes through two distinct scoring passes. The pre-evidence candidates (v1) are ranked on structure, timing, and telemetry alone. The post-evidence candidates (v2) are re-ranked after the evidence bundle is retrieved. The delta between v1 and v2 ranking is the most important diagnostic signal for whether evidence retrieval is providing discriminating value.

3. System Architecture and Interfaces

The current implementation uses Neo4j for graph traversal and Chroma for vector retrieval. The orchestrator stitches together typed stage outputs rather than free-form intermediate text. This keeps the method inspectable and simplifies validation.

Module	Reads	Produces
Ingestion / snapshot	Event stream, telemetry store, ops logs, CMMS/EAM	event.json, telemetry_summary.json, optional operational_context.json, pm_compliance.json
Neo4jKGContextBuilder	Neo4j + event/telemetry context	kg_context.json
TSKRTemporalScorerV1	event, telemetry_summary, kg_context	tskr_patterns.json
RuleBasedCausalityEngineV32	kg_context, telemetry_summary, tskr_patterns, pm_compliance, op_context	causality_candidates.json v1 (with recurrence + common-cause)
ChromaEvidenceRetriever	kg_context, causality_candidates v1, op_context	evidence_bundle.json
RuleBasedCausalityEngineV32.refine_with_evidence()	causality_candidates v1, evidence_bundle	causality_candidates.json v2
HeuristicIshikawaEvaluatorV1	all prior artifacts	ishikawa_matrix.json
RuleValidatedRCASynthesizerV31	all prior artifacts + optional LLM client	rca_card.json
RCAArtifactValidator	all artifacts	validation reports
FileArtifactStore	validated artifacts	persisted JSON files per run_id

Module	Reads	Produces
Analyst review / write-back	validated RCA card + run_manifest	Accepted RCA edges/nodes + archived artifact bundle

LLM Client Status

Two LLM clients are implemented:

- **DummyLLMClient** — development stub that intentionally raises, forcing the deterministic fallback path. Currently the default in `build_dev_orchestrator()`.
- **OllamaLLMClient** — real implementation using the Ollama `/api/generate` endpoint with `format: json`. Exists in the codebase but not yet validated end-to-end against the full schema contract.

The deterministic fallback path in `RuleValidatedRCASynthesizerV31._fallback_card()` is the main output: it deterministically performs confidence calibration, recurrence interpretation, common-cause interpretation, evidence posture derivation, and analyst review question generation.

4. Formal Data-Model Expectations

The canonical machine contract lives in the JSON schemas under `schemas/`. Two important schema consistency issues are currently open and must be resolved before the next iteration:

Issue — confidence label case mismatch: `causality_candidates.json` (runtime schema) uses lowercase confidence labels ("high", "medium", "low", "speculative"). `causality_candidates.v3_2.schema.json` uses uppercase ("HIGH", "MEDIUM", "LOW", "SPECULATIVE"). The causality engine v32 produces lowercase. The schema validator will reject output depending on which file it loaded. **Resolution required before next release.**

Issue — kg_context dual schema: The v2 `kg_context.json` schema has a different `upstream_paths` structure than the runtime `kg_context` produced by `Neo4jKGContextBuilder`. The orchestrator currently runs with `stop_on_validation_error=False` to tolerate this. **Resolution required for strict validation runs.**

Artifact	Key Required Fields	Notes
event.json	event_id, timestamp_start, asset_id, severity, event_type, symptom_signature	severity and event_type follow uppercase enumerations. component_id optional at intake
telemetry_summary.json	event_id, asset_id, generated_at, window, signals[]	Signals carry parameter, unit, stats, anomalies[], changepoints[], data_quality
kg_context.json	event_id, asset_id, subgraph_id, components, failure_modes, upstream_paths, past_events, documents, seed_context	Two schema versions exist (v1 and v2). Runtime output includes asset_id, subgraph_id, generated_at, hop_limit beyond strict v2 schema
tskr_patterns.json	event_id, asset_id, patterns[], summary, provenance	Patterns carry relation, mean_lag_hours, latency_alignment_score, latency_violation_type, temporal_contradiction, support, confidence
causality_candidates.json	event_id, scoring_config, screening, summary,	V32 adds recurrence_summary, common_cause_summary. Candidates

Artifact	Key Required Fields	Notes
	candidates[], filtered_out_candidates[]	carry evidence_posture, temporal_posture, recurrence{}, common_cause{} after v32 retrieval_scope must be explicit about event and asset scope.
evidence_bundle.json	bundle_id, generated_at, retrieval_scope, results[], candidate_evidence_summary[]	candidate_evidence_summary is the key consumer input for evidence refinement.
ishikawa_matrix.json	matrix_id, event_id, asset_id, categories[], summary, provenance	Rows grouped by category and reference source artifacts and linked_candidate_ids.
rca_card.json	rca_id, event_id, generated_at, llm_model, validation_status, executive_summary, primary_hypothesis, evidence[], recommended_actions[], analyst_review, provenance	primary_hypothesis requires why_primary[], uncertainties[], citations[]. All claims must be cited.
run_manifest	run_id, completed_at, artifacts{}, validation{}, review_hooks{}	review_hooks.writeback_ready is currently always False because fallback_used=True. review_hooks.requires_human_review is always True.

5. Knowledge Graph Design

The KG is the structural backbone of the RCA method. It defines the causal search space in Stage 1 and preserves accepted RCA conclusions for future recurrence analysis after analyst write-back.

Node / Edge	Illustrative Properties	Purpose
:asset	asset_id, name, type	Top-level plant asset (e.g., pump, condenser).
:mbse_entity	id, name, type, role	Plant component from the MBSE model. Seed of the causal search.
:failure_mode	fm_id, name, mechanism, expected_latency_min_hours, expected_latency_max_hours	Failure semantics used by the causality engine. The FMEA taxonomy lives here.
:abnormal_event	id, asset_id, timestamp_start, severity, event_type	Anchors investigated events and historical analogs for recurrence.
:monitored_variable	ID, variable, sensor_id, tag_id, source_system	Links telemetry signals to components. Seed resolution uses four lookup strategies with confidence ranking.

Node / Edge	Illustrative Properties	Purpose
:document / :condition_report / :work_order	doc_id, doc_type, created_at, revision	Textual evidence connected to components and assets.
:ProcessedTextRecord	record_id, doc_id, doc_type, chunk_index, embedding_text	Unit of Chroma storage. One per document chunk.
UPSTREAM_OF / CONNECTED_TO / CONTAINS	path_strength	Structural edges traversed during neighborhood expansion.
REALIZES	—	Connects asset to its realized mbse_entity components.
MONITORS / MEASURES	—	Connects monitored_variable to mbse_entity.
CONFIRMED_CAUSE / MAY_CAUSE	confidence, evidence_refs, analyst_provenance	Written back after analyst acceptance. Preserves accepted causal knowledge.

KG Governance — Open Issues

The following governance questions are unresolved and must be addressed before production deployment:

1. **Initial population:** Where does the KG come from? Plant MBSE models, vendor data, or manual entry? What is the seeding process for a new unit?
2. **FMEA taxonomy maintenance:** Who owns the failure mode taxonomy? On what cadence is it updated? Who resolves new failure modes identified during RCA?
3. **Currency after plant modifications:** When an engineering change notice modifies a system boundary or installs a new component, what is the process for updating the KG before the next event analysis?
4. **Unresolved entity alias review:** The NLP pipeline flags unresolved entity aliases (surface forms that could not be mapped to canonical KG nodes). Who reviews these, and on what cadence? Currently: no defined process.

6. Candidate Scoring and Confidence Model

Pre-Evidence Scoring (Stage 3 — V32)

Each candidate is scored across five dimensions before any documents are retrieved:

Dimension	Meaning	Source
Structural	Physical connectivity of the failure mode's component to the event asset	KG neighborhood, seed_match_type, hop distance
Temporal	Anomaly timing, latency alignment, TSKR pattern confidence	tskr_patterns, telemetry_summary
Telemetry	Anomaly burden and severity across monitored signals	telemetry_summary.signals[].anomalies[]
Evidence prior	Document type presence in scope (not retrieval — pre-retrieval prior)	kg_context.documents[].doc_type

Dimension	Meaning	Source
Governance	PM compliance posture (currently asset-level, not candidate-specific — known gap)	pm_compliance.summary, checks[]

Known gap — governance uniformity: The governance score is currently computed at the asset level and applied uniformly to all candidates regardless of whether the failed PM item is related to the specific failure mode being scored. This must be replaced with candidate-specific PM linkage in the next iteration.

Known gap — symptom matching: The `event.symptom_signature.symptom_types` and `anomaly_pattern` fields, and the corresponding `kg_context.failure_modes[].expected_symptoms` and `expected_anomaly_pattern` fields, are not currently used in scoring. A failure mode that expects `gradual_drift` and whose component shows `gradual_drift` anomalies should receive a structural bonus. This is unimplemented.

Known gap — weights not normalized: No constraint enforces that scoring weights sum to 1.0. A configuration change to one weight without adjusting others will silently produce an unnormalized composite score.

Post-Evidence Refinement (Stage 5 — V32)

After evidence retrieval, the evidence score is refined using:

```
refined_evidence = clamp(
    0.30 * prior_evidence
    + 0.55 * best_support_score
    + 0.15 * best_context_score
    - 0.45 * best_contradiction_score,
    0.0, 1.0
)
```

The composite score is recomputed using the same weights with the refined evidence score. Candidates are re-sorted and re-screened against thresholds. If only one candidate passes, the best failed candidate is evaluated for retention as a review alternative if its score gap is within `review_alternative_gap` (default 0.10) and it is not temporally contradicted.

Confidence Calibration (Stage 7 — Synthesizer)

The final `confidence_label` on the primary hypothesis is **not** a simple threshold on the composite score. It is computed by `_calibrate_primary_confidence()` using:

- Supporting evidence count and score
- Contradicting evidence count and score
- Evidence posture label (supported / mixed / contextual_only / contradicted / weak)
- Score gap to runner-up candidate
- Temporal posture (supported / partial / contradicted / weak)
- Temporal contradiction flag
- Recurrence score and confidence
- Common-cause score and confidence
- Whether the fallback path was used (caps at medium if True)

This multi-dimensional calibration is intentional. A candidate with a high composite score but zero supporting evidence and a temporal contradiction must not be labeled `high` confidence. The calibration model enforces this constraint.

7. Formal Workflow Steps

Step A — Run Setup and Input Validation

Class: `RCARReasoningOrchestrator._stage_a_build_run_context()`

Validate the incoming event and telemetry summary against their JSON schemas and semantic rules. Establish the `run_id`, pipeline configuration, and input validation result. Persist `run_context`. This stage does not perform causal reasoning.

Step B — KG Context Construction

Class: `Neo4jKGContextBuilder.build()`

Resolve seed components from the event asset and telemetry signal identifiers (using four lookup strategies: `monitored_variable_id`, `sensor_id`, `tag_id`, `ID`, aliases — with confidence ranking and deduplication). Expand the graph neighborhood within `max_hops`. Fetch failure modes, ranked documents, and ranked past events. The KG context defines the causal search space for all subsequent stages.

Step C — Temporal Reasoning (TSKR)

Class: `TSKRTemporalScorerV1.score()`

For each failure mode in the KG context, compute: temporal relation (precedes / simultaneous / follows / unknown), mean lag and std lag against anomaly timestamps, latency alignment score against expected FMEA latency bounds, latency violation type (none / too_fast / too_slow / unknown), temporal contradiction flag, history support score from past event overlap, and combined pattern confidence and support scores. Currently heuristic — the scorer's weakest stage.

Known gap — TSKR index limitation: The TSKR index keeps only the first pattern per `target_id`. Failure modes with multiple TSKR patterns at different latencies lose all but the first. Fix required: store a list and select highest-confidence pattern, or aggregate.

Known gap — anomaly timestamp weighting: All anomaly timestamps are treated as equal regardless of severity or signal quality. High-severity anomalies should contribute more to lag estimation than low-severity or noisy ones.

Step D — Candidate Generation (Pre-Evidence)

Class: `RuleBasedCausalityEngineV32.generate()`

Generate failure mode and historical event candidates. Score each across five dimensions. Enrich with recurrence features (from `past_event_index`) and common-cause features (from `common_cause_index`). Apply dual threshold screening (composite ≥ 0.30 AND evidence ≥ 0.35). Retain top-k. Store filtered candidates with filter reason. Compute `recurrence_summary` and `common_cause_summary`. Output is `causality_candidates v1`.

Note: V31 (`RuleBasedCausalityEngineV31`) remains in the codebase for reference and backward compatibility but should not be used for new runs. V32 is the production engine.

Step E — Evidence Retrieval

Class: `ChromaEvidenceRetriever.retrieve()`

Build hypothesis-guided query plans for each top candidate: one support query and one contradiction query per candidate, plus failure mode context, component context, and operational context queries. Execute each query against the Chroma vector store with `asset`, `doc_type`, `doc_id`, and component filters. Score each retrieved snippet for support, contradiction, and context roles using keyword cue detection, candidate term overlap, authority weight, and extraction quality. Deduplicate and rank. Build per-candidate evidence summary (supporting counts, contradiction counts, best scores, snippet IDs). Output is `evidence_bundle`.

Known gap — component_ids filter is a no-op: The Chroma metadata structure does not support direct list-valued filter on component_ids. The filter is built but silently ignored. All documents for the asset are searched regardless of component scoping. This reduces retrieval precision for assets with many components.

Known gap — BM25 unavailable on disk-loaded collections: When collections are loaded from disk (not ingested in the same process), state.bm25_docs is empty and hybrid retrieval degrades silently to dense-only. No warning is emitted. Fix required: add explicit BM25 availability flag in retrieval metadata.

Known gap — keyword-based role classification: Support, contradiction, and context role classification uses keyword tokenization against snippet text. Semantically equivalent terms ("loss of lubrication" vs "lube oil degradation") will not match. This is a known precision limitation for the nuclear terminology domain.

Known gap — document type semantics not differentiated: All document types (CR, WO, SOP, ECA, RCA, FMEA) pass through the same keyword classifier. A SOP's conditional statement ("fouling may cause backpressure rise") scores identically to a confirmed WO finding. The structured fields in the document schema (condition_assessment.as_found_condition, failure_mode_refs, extracted_causal_statements) are not used in role classification. This is one of the highest-priority improvements for retrieval quality.

Step F — Candidate Refinement (Post-Evidence)

Class: RuleBasedCausalityEngineV32.refine_with_evidence()

For each retained candidate, update the evidence score using the refinement formula. Recompute composite score and threshold pass/fail. Derive evidence_posture label. Re-sort and re-screen candidates. Evaluate review alternative rescue if only one candidate passes. Mark evidence_refinement_applied: True in provenance. Output is causality_candidates v2.

This is architecturally the most important stage. The delta between the Stage D ranking and the Stage F ranking is the system's primary discriminating contribution. If the ranking does not change after evidence refinement, evidence retrieval is not providing useful information and should be diagnosed.

Step G — Ishikawa Structuring

Class: HeuristicIshikawaEvaluatorV1.evaluate()

Translate all prior reasoning into five engineering RCA categories: equipment_hardware, process_procedure, measurement_instrumentation, environment_operating_context, maintenance_human_factors. This stage performs translation, not new inference. Every row carries links to its source artifact, linked candidate IDs, supporting evidence IDs, and telemetry signals. Output is ishikawa_matrix.

Known gap — Ishikawa depth: The evaluator is heuristic. Row strength values are approximations. The process_procedure category is populated from top evidence snippets rather than from actual procedural deviation analysis. Human performance and organizational factor analysis (required by IAEA TECDOC-1756 and INPO AP-923) is not implemented.

Step H — RCA Synthesis

Class: RuleValidatedRCASynthesizerV31.synthesize()

Select top candidates and top evidence. Attempt LLM structured generation (currently always fails → fallback). Build deterministic RCA card from the ranked candidates and evidence bundle. Derive executive summary, primary hypothesis with why_primary[] and uncertainties[], alternatives with supports/weaknesses/citations, evidence rows with support_role, recommended actions, and analyst review questions. Apply multi-dimensional confidence calibration. Set validation status flags. Output is rca_card.

Current production path: Deterministic fallback (`_fallback_card()`). The LLM path (`OllamaLLMClient`) exists in the codebase but is not validated.

Known gap — LLM hallucination gate: If the LLM path is used and produces a hallucinated candidate ID that passes schema validation, the cross-artifact check in the validator will catch it as a warning (not an error). This should be upgraded to a hard error that triggers the fallback path.

Step I — Artifact Persistence

Class: `FileArtifactStore`

Write all stage outputs as JSON files under `{output_dir}/{run_id}/`. Validation reports are persisted alongside each artifact.

Step J — Validation and Run Manifest

Classes: `RCAArtifactValidator.validate_run_bundle()`, `RCAReasoningOrchestrator._stage_g_finalize_manifest()`

Run two-layer validation: JSON schema checks per artifact, and cross-artifact semantic checks (event_id / asset_id consistency across all artifacts, subgraph_id consistency, primary candidate ID exists in candidates, cause_label matches, evidence source IDs exist in bundle, candidate count arithmetic consistency, temporal posture vs contradiction flag consistency). Compute review hooks. Output is `run_manifest`.

`requires_human_review:` True always. `writeback_ready:` False always in current implementation because `fallback_used:` True.

8. RCA Workflow — File-to-Stage Map

Stage	Description	Main File	Main Class / Function	Output Artifact
A	Run context initialization	<code>rca_reasoning_orchestrator.py</code>	<code>_stage_a_build_run_context()</code>	<code>run_context</code>
B	KG context construction	<code>rca_reasoning_orchestrator.py</code>	<code>Neo4jKGContextBuilder.build()</code>	<code>kg_context</code>
C	Temporal reasoning (TSKR)	<code>tskr_temporal_scorer.py</code>	<code>TSKRTemporalScorerV1.score()</code>	<code>tskr_patterns</code>
D	Candidate generation (pre-evidence)	<code>causality_engine_v32.py</code>	<code>RuleBasedCausalityEngineV32.generate()</code>	<code>causality_candidates v1</code>
E	Evidence retrieval (Chroma)	<code>evidence_retriever.py</code>	<code>ChromaEvidenceRetriever.retrieve()</code>	<code>evidence_bundle</code>
F	Candidate refinement (post-evidence)	<code>causality_engine_v32.py</code>	<code>RuleBasedCausalityEngineV32.refine_with_evidence()</code>	<code>causality_candidates v2</code>
G	Ishikawa structuring	<code>rca_reasoning_orchestrator.py</code>	<code>HeuristicIshikawaEvaluatorV1.evaluate()</code>	<code>ishikawa_matrix</code>
H	RCA synthesis	<code>rca_synthesizer_v31.py</code>	<code>RuleValidatedRCASynthesizerV31.synthesize()</code>	<code>rca_card</code>
I	Validation	<code>schema_validator.py</code>	<code>RCAArtifactValidator.validate_run_bundle()</code>	validation reports
J	Artifact persistence + run manifest	<code>rca_reasoning_orchestrator.py</code>	<code>FileArtifactStore, _stage_g_finalize_manifest()</code>	stored artifacts, <code>run_manifest</code>

Supporting Components

Functionality	File	Class
Orchestration	<code>rca_reasoning_orchestrator.py</code>	<code>RCAReasoningOrchestrator</code>
KG access (Neo4j wrapper)	<code>py2neo_workflow.py</code>	<code>Py2Neo</code>
Validation	<code>schema_validator.py</code>	<code>RCAArtifactValidator</code>
Chroma store	<code>chroma_store.py</code>	<code>ChromaRecordStore</code>
Hybrid retrieval	<code>lc_retriever_processed.py</code>	<code>LCProcessedRetriever</code>

Functionality	File	Class
Evidence store adapter	processed_evidence_store_adapter.py	ProcessedEvidenceStoreAdapter
Processed record hydration	processed_record_store.py	ProcessedRecordStore
Multi-view fusion	multi_vector_fusion.py	reciprocal_rank_fusion, weighted_distance_inversion
KG ingest	kg_ingest_neo4j_workflow.py	ingest_workflow_case_to_neo4j()
KG schema builder	kg_schema_builder_workflow.py	build_graph_from_workflow_artifacts()
Dev LLM fallback	rca_reasoning_orchestrator.py	DummyLLMClient
Real LLM client	rca_reasoning_orchestrator.py	OllamaLLMClient
Factory function	rca_reasoning_orchestrator.py	build_dev_orchestrator()

9. Current Implementation Status

Core RCA Workflow

Stage	Component	Status	Notes
KG Context	Neo4jKGContextBuilder	Implemented	Runtime-aligned; dual schema issue open
Temporal (TSKR)	TSKRTemporalScorerV1	Heuristic	Deterministic; single-pattern-per-target limitation; anomaly weighting missing
Candidate Engine (pre-evidence)	RuleBasedCausalityEngineV32.generate()	Enhanced	Recurrence + common-cause integrated; governance uniformity gap open
Evidence Retrieval	ChromaEvidenceRetriever	Basic	Functional; component filter no-op; keyword-only classification; doc-type semantics not differentiated
Candidate Refinement (post-evidence)	RuleBasedCausalityEngineV32.refine_with_evidence()	Implemented	Two-pass architecture working; review alternative rescue working
Ishikawa	HeuristicIshikawaEvaluatorV1	Heuristic	Structure good; human performance / organizational factors not implemented
RCA Synthesis	RuleValidatedRCASynthesizerV31	Enhanced	Deterministic fallback is production path; confidence calibration multi-dimensional
Orchestrator	RCAReasoningOrchestrator	Monolithic	End-to-end stable; file size is technical debt; split required
Validation	RCAArtifactValidator	Implemented	Full schema + cross-artifact coverage; three modes (strict/compat/warn_only)
Confidence Calibration	_calibrate_primary_confidence()	Implemented	Multi-dimensional; caps at medium when fallback used

Artifacts

Artifact	Status	Notes
event	Stable	Schema aligned
telemetry_summary	Stable	Strict schema enforced
kg_context	Transitional	Dual schema issue open (v1 vs v2 upstream_paths structure)
tskr_patterns	Expanding	Latency fields added; additionalProperties: true (inconsistent with other schemas)
causality_candidates	Schema conflict	Two schema files with uppercase vs lowercase confidence labels — must resolve
evidence_bundle	Stable	candidate_evidence_summary working
ishikawa_matrix	Heuristic	Schema valid; depth limited
rca_card	Enhanced	Calibrated confidence; recurrence + common-cause surfaced; LLM path not validated
run_manifest	Stable	writeback_ready always False in current implementation

Infrastructure

Component	Status	Notes
Chroma integration (ChromaRecordStore)	Functional	BM25 unavailable on disk-loaded collections; component filter no-op
Hybrid retrieval (RRF + WDI)	Implemented	Both fusion strategies working
Neo4j integration	Stable	Custom Py2Neo wrapper; batch upsert working
LLM client (OllamaLLMClient)	Implemented, not validated	Code exists; not tested against full schema contract
LLM client (DummyLLMClient)	Stable	Intentionally raises to force deterministic fallback
Parsing/NLP pipeline	Out of scope	Deferred; ProcessedTextRecord schema exists but pipeline not implemented
CAP integration	Not started	recommended_actions in rca_card.json are not connected to Maximo/SAP PM

10. Known Gaps and Prioritized Technical Debt

Priority 1 — Must Fix Before Next Release

#	Area	Issue	Impact
1	Schema	confidence_label case mismatch between causality_candidates.json and causality_candidates.v3_2.schema.json	Validator rejects valid output depending on which schema file is loaded
2	Schema	kg_context dual schema — v1 vs v2 upstream_paths structure	Requires stop_on_validation_error=False; prevents strict validation runs

#	Area	Issue	Impact
3	Causality Engine	Governance score is asset-level, not candidate-specific	Failed PM items for expansion joint inspection boost tube fouling candidates equally — reduces analytical value
4	KG Schema Builder	causes edge type overloaded across six semantically distinct relationships	KG unusable for path-strength analysis; cannot distinguish alarm context from confirmed root cause
5	TSKR	TSKR index keeps only first pattern per target_id	Failure modes with multiple patterns lose all but the first

Priority 2 — High Value for Next Iteration

#	Area	Issue	Impact
6	Causality Engine	Symptom-to-failure-mode matching not implemented	event.symptom_signature fields and failure_mode.expected_symptoms fields not used in scoring
7	Evidence Retrieval	component_ids filter is a no-op in Chroma	All asset documents searched regardless of component scoping; reduces retrieval precision
8	Evidence Retrieval	BM25 unavailable on disk-loaded collections with no warning	hybrid_weight setting silently becomes meaningless
9	Evidence Retrieval	Document type semantics not differentiated in role classification	SOP conditional statements score identically to confirmed WO findings
10	Schema	scoring_config.weights has no sum-to-1.0 constraint	Weight changes silently produce unnormalized composite scores
11	LLM Path	Hallucinated candidate ID triggers warning not hard error	Hallucinated primary hypothesis could pass schema validation
12	Synthesizer	_passes_minimum_evidence_gate uses bare string equality	Whitespace or type coercion artifacts cause gate to silently fail

Priority 3 — Architecture and Maintainability

#	Area	Issue	Impact
13	Orchestrator	rca_reasoning_orchestrator.py is monolithic	Contains orchestrator, all concrete implementations, and factory function — harder to test and maintain
14	Analyst Override	No structured artifact for analyst override records	Cannot mine override history for recurrence signal; regulatory auditability gap
15	CAP Integration	No connection from recommended_actions to Maximo/SAP PM	Engineers must manually re-enter findings; adoption risk
16	TSKR	Anomaly timestamps not weighted by severity	Low-severity noise anomalies contribute equally to lag estimation
17	KG Schema Builder	_prefix() can produce IDs with spaces	Neo4j IDs with spaces cause Cypher issues in unquoted contexts

#	Area	Issue	Impact
18	Evidence Retrieval	Time_distance_days in kg_context.documents not used in evidence prior scoring	Recent CRs and old SOPs treated identically in evidence prior

Deferred (Not in Scope for Next Iteration)

- Human performance analysis (task analysis, error precursor identification, organizational factor assessment)
- Precursor detection (weak signal aggregation before event)
- NLP/NER parsing pipeline for ProcessedTextRecord population
- Fleet-wide recurrence (cross-unit, cross-plant similarity)
- PRA / risk significance integration
- Effectiveness review tracking

11. Acceptance Criteria and Developer Guidance

Reproducibility: Identical inputs must produce identical `causality_candidates` and validation outcomes. The pipeline is fully deterministic when `DummyLLMClient` is used.

Grounding: Primary causal claims in the RCA card must reference evidence or be explicitly marked low-confidence/speculative. `all_claims_cited` must be `True` before write-back is considered.

Auditability: Every artifact carries provenance (`generated_at`, `generated_by`, `run_id`, `pipeline_version`). Every KG commit carries analyst sign-off context. The `run_manifest` is the permanent audit record.

Explainability: The RCA result must be defensible through ranked candidates, temporal evidence, Ishikawa categories, and evidence citations — not only by narrative summary. The `score_rationale` block on each candidate must be populated.

Schema hygiene: Resolve confidence label case mismatch and `kg_context` dual schema before enabling strict validation. All schemas must have `additionalProperties: false` for deterministic validation behavior.

Repository hygiene: Schemas in a single discoverable directory (`schemas/`). Example fixture bundles synchronized with schemas. V31 causality engine retained for reference but clearly marked as deprecated in favor of V32.

12. Reference Test Case

Primary validation scenario: PWR Unit 2 — Condenser Vacuum Loss / Turbine Load Runback (EVT-U2-2024-0847)

True root cause: Air in-leakage through turbine exhaust duct expansion joint
Contributing cause: HVAC fan bearing failure → elevated pit ambient temperature → accelerated expansion joint thermal fatigue

Red herring: Condenser waterbox tube cleaning 21 days prior (found acceptable)

Recurrence trap: Most recent similar event (18 months prior) had fouling as confirmed root cause — recurrence scorer must not over-weight recency

Key discriminating signal: Hotwell dissolved oxygen = 142 ppb (normal < 10 ppb). DO elevation is caused by air in-leakage and is NOT produced by tube fouling. The condenser tube inspection WO explicitly states "within normal limits, zero tubes plugged" — this is the primary contradicting evidence for the fouling hypothesis.

Assertion Targets

Assertion	Expected Behavior	Failure Indicators
A1	Primary hypothesis maps to FM-CND-AIR-INLEAK	Evidence refinement not working
A2	FM-CND-TUBE-FOUL in alternatives, not primary	WO contradiction not classified correctly
A3	FM-CND-TUBE-LEAK filtered or temporal_contradiction=True	Latency alignment working (2-48h vs 336h drift)
A4	WO-2024-11847 classified as contradicting for fouling candidate	Keyword classifier missing "within normal limits"
A5	Score gap between #1 and #2 > 0.05 after refinement	Evidence refinement not discriminating
A6	Air in-leakage recurrence_score ≥ fouling recurrence_score	Recency bias in recurrence scorer
A7	FM-CW-TEMP-RISE not primary; operator "minimal effect" note classified as contradicting	Operational context query working
A8	FM-HVAC-DEGRAD present in Ishikawa maintenance_human_factors category	HVAC temporal chain being captured
A9	Analyst review questions reference expansion joint inspection deferral	PM governance linking to specific candidates
A10	rca_card schema_valid=True AND all_claims_cited=True	Schema compliance and citation completeness

13. Next Iteration Priorities

Must Do

1. Resolve confidence_label case mismatch (uppercase vs lowercase) across schema files
2. Resolve kg_context dual schema — pick v2 as canonical, update KGContextBuilder output
3. Add sum-to-1.0 constraint on scoring_config.weights in schema and validator
4. Fix TSKR index to store all patterns per target_id, select highest-confidence
5. Make LLM hallucinated candidate ID a hard validation error (not warning)
6. Refactor rca_reasoning_orchestrator.py — split KGContextBuilder, IshikawaEvaluator, factory function into separate modules

High Value

7. Implement candidate-specific governance scoring — link failed PM items to specific failure mode candidates by component and check_type
8. Add symptom-to-failure-mode matching using symptom_types and anomaly_pattern fields
9. Fix component_ids filter in Chroma — or document explicitly as no-op with warning
10. Add BM25 availability flag to retrieval metadata with explicit warning on degradation
11. Differentiate document type role classification — use condition_assessment.as_found_condition for WO classification; treat SOP diagnostic rules as discriminating logic rather than supporting evidence
12. Add time_distance_days weighting to evidence prior scoring in the causality engine
13. Fix _passes_minimum_evidence_gate string comparison to use .strip() and explicit casting
14. Add analyst_override schema and ingestion pathway

Defer

- NLP/NER parsing pipeline
- Human performance analysis tools
- Precursor detection
- PRA integration
- CAP integration (Maximo/SAP PM)
- Production deployment concerns

14. Notes on TSKR

Nuclear plant telemetry is exactly the kind of noisy, multi-variate, interval-based time series that TSKR was designed for. The hierarchical Tone → Chord → Phrase structure maps naturally onto plant failure propagation:

- **Tones** → discretized parameter states ("lube oil pressure low," "vibration elevated," "temperature rising")
- **Chords** → coincident degradation signatures (multiple parameters anomalous simultaneously)
- **Phrases** → temporal sequences of failure propagating through a system

Use TSKR for: justifying the representation choice over Allen's relations; the Tone discretization approach for converting continuous plant parameters into symbolic interval states; the margin-closedness concept for pruning temporal pattern results.

Don't rely on TSKR for: full mining pipeline implementation, lag/confidence modeling, or event-relative temporal scoring — these are gaps that require additional domain engineering work.

The most important next step for making TSKR useful is defining the Tone discretization vocabulary for nuclear plant parameters — what thresholds and durations make a "lube oil low" Tone versus a transient fluctuation. This is domain engineering work that no paper will provide, and it is the foundation everything else rests on. The FMEA `expected_latency_min_hours` and `expected_latency_max_hours` fields in the KG are the starting point for this vocabulary.

15. Open Issues Tracker

#	Issue	Status	Priority
1	Graceful degradation when KG returns empty subgraph	✅ Resolved	—
2	Recurrence and common-cause analysis	✅ Resolved (precursor detection remains future work)	—
3	CAP integration (Maximo/SAP PM)	❌ Not resolved	High
4	Confidence rationale explainability	🟡 Partially resolved (multi-dimensional calibration implemented; machine-readable rationale block still recommended)	Medium
5	KG governance (initial population, FMEA maintenance, currency after mods, alias resolution)	❌ Not resolved	Critical for production
6	Confidence label case mismatch in schemas	❌ Not resolved	High — blocks strict validation
7	Governance score uniformity (asset-level not candidate-specific)	❌ Not resolved	High

#	Issue	Status	Priority
8	Symptom-to-failure-mode matching not implemented	✗ Not resolved	High
9	Document type semantics not differentiated in evidence classification	✗ Not resolved	High
10	Analyst override artifact schema missing	✗ Not resolved	Medium
11	<code>causes</code> edge type overloaded in KG schema builder	✗ Not resolved	High
12	TSKR single-pattern-per-target-id limitation	✗ Not resolved	High

Gaps from Requirements document

Gap 1 — `kg_snapshot_version` never populated (§6 Model Governance) [Fixed]

The schema declares it as "Required for reproducibility — RCA must be replayable against the KG state that existed at event time." `Neo4jKGContextBuilder.build()` never queries Neo4j for a snapshot version or timestamp and never sets the field. Every run produces a `kg_context` that silently omits it, making the output non-reproducible against KG changes — which is a regulatory defensibility problem.

Fix: Query `CALL dbms.components() YIELD versions` or `CALL apoc.meta.stats()` at build time and store the result, or at minimum stamp `generated_at` as the snapshot version.

Gap 2 — `recent_alarms` is ingested but never used in causal scoring (§3.3) [Fixed]

`operational_context.recent_alarms` carries `alarm_id`, `priority`, `system_affected`, `setpoint`, `actual_value`. The causal engine has zero alarm handling — no field is read, no score is adjusted. A **critical** alarm on the same component as a candidate is strong corroborating structural signal; a low-priority alarm on a different system is irrelevant noise. The engine currently treats all alarm states identically: ignored.

Fix: In `_structural_score_for_fm`, extract alarms from `operational_context` and boost structural score when a critical/high alarm's `system_affected` matches the candidate's component or its upstream.

Gap 3 — Defense-in-depth barrier analysis completely absent (§3.2, §3.12)

The document makes barrier analysis a first-class requirement: "which barriers failed, which succeeded." This is distinct from safety functions — barriers are the physical/functional layers (fuel cladding, RCS pressure boundary, containment) that prevent radioactive release. Safety function mapping (Issue #8 we just implemented) is one step, but there is no barriers concept in:

- `kg_context.json` schema
- Neo4j KG context builder
- Candidate scoring
- `rca_card` recommended actions

The document also calls for "defense-in-depth barrier identification" and "visualization of degraded defense layers" explicitly in §3.2 and "identification of barrier weaknesses" in §3.12. This is a structural gap in the architecture — the KG schema doesn't model barriers at all.

Gap 4 — No risk significance score derived from safety function impact (§3.12)

affected_safety_functions on each candidate tells you *which* safety functions could be affected, but there's no risk_significance scalar derived from it. A candidate affecting "Core Cooling" is orders of magnitude more safety-significant than one affecting "Instrument Air." The document requires "qualitative risk significance scoring" as part of PRA integration. Currently the field is populated but never used to adjust confidence or flag candidates for priority review.

Gap 5 — mdParser.py is missing doc types and section roles (§3.1, §3.7)

FIELD_LABEL_MAP covers CR, WO, SOP, ECA, OTHER — but:

- **No FMEA** section role map — the parser has no concept of FMEA-specific sections (failure mechanism, detection method, mitigation). FMEA chunks are parsed as OTHER, losing structured signal. [Fixed]
- **No INPO OE / fleet-level documents** — the document calls for "industry OE reports" and "operating experience repositories" as a primary data source (§3.7). There is no "OE" or "INPO" entry in FIELD_LABEL_MAP. [Fixed]
- **No protection logic extraction** — no section roles for permissive/interlock/trip references in any doc type, which §3.2 requires.
- **No alarm reference extraction** — alarm IDs mentioned in CR narratives or operator logs are not tagged. [should be fixed now]

Gap 6 — governance score rationale is non-traceable (§3.8, §7) [Fixed]

The score_rationale["governance"] on every candidate says "Governance score derived from PM compliance signals." — a generic string that never identifies *which* PM checks matched, what keywords triggered them, or what the check result was. The document requires "audit trail from raw data to final conclusion" and "evidence traceability for every inference." The current rationale doesn't satisfy this for governance.

Gap 7 — Change analysis is unstructured (§3.8)

The document requires a "change analysis workflow" as a named RCA methodology.

operational_context.nearby_maintenance is used in CCF scoring, but there is no structured change analysis that systematically correlates:

- Maintenance completed in the window
- Setpoint/procedure revisions
- Configuration changes (ECNs)

...against the event timeline. Currently maintenance proximity influences common_cause scoring heuristically, but it's not surfaced as a named "change analysis" artifact in the output the engineer reviews.

Gap 8 — Sensitivity analysis missing (§3.9)

All candidate scores are point estimates. The document explicitly requires "sensitivity analysis under data-quality assumptions." If extraction_quality on a key piece of evidence went from 0.6 to 0.9, how much does the composite score change? Which candidates are score-stable vs. fragile? This is never computed or surfaced. The evidence_gap flag (implemented this session) covers the zero-evidence case, but not the quality-sensitivity case.

Gap 9 — No FMEA doc type in mdParser.py and no FMEA-specific structured output feeding the document schema [FMEA parser added]

FMEAs carry failure_mechanism, rpn, expected_latency — all fields that the kg_context.json failure_modes schema defines. But the parser has no FMEA section map, so FMEA documents are

chunked as generic text. The structured fields in `failure_modes` in `kg_context` must currently be manually populated in the KG rather than being extracted by the parser.

Gap 10 — Visualize timing of events and RCA retrieved data

There must a way to visualize present information retrieved and generated by the RCA workflow. This should be interactive

On OE documents

OE reports (INPO OE, NRC Information Notices, Generic Letters, EPRI Technical Reports, fleet-wide SOER/SOER) are fundamentally different from plant-internal documents in three ways:

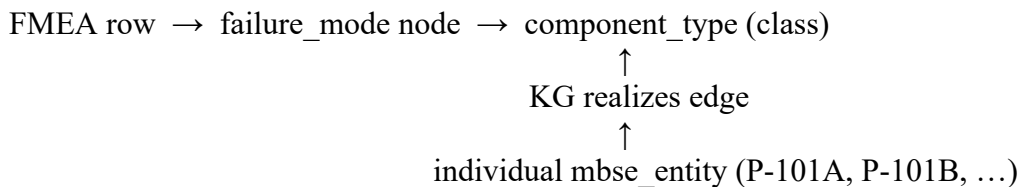
1. Scope is fleet-wide, not asset-specific. A CR is tied to a specific equipment tag at a specific plant. An INPO OE report says "this failure mode has occurred at 7 plants on this component family." There is no `asset_id` to resolve, no `component_id` — the linkage is through failure mode similarity and system type, not plant topology.
2. Authority and epistemic role are distinct. OE is guidance/informational — it can't confirm your specific event's cause. Its value is plausibility amplification: "this has happened before at similar plants." That's different from an ECA which confirms a cause at this plant.
3. Temporal semantics are different. OE from 2005 about valve stiction degradation is still fully relevant today — it's closer to FMEA (timeless engineering knowledge) than to a CR (time-bound operational observation). Recency decay should not apply.

On FMEA documents

FMEA is class-level, not instance-level. This has important consequences:

- A single FMEA row for "centrifugal pump / seal degradation" applies to every centrifugal pump in the plant
- The FMEA says nothing about which specific pump P-101A will fail — only that pumps of that type can fail that way
- Individual pump P-101A is a KG `mbse_entity` that realizes the `component_type = centrifugal_pump`

The mapping chain is therefore:



This means the FMEA parser cannot write `applies_to_component_id = P-101A` directly — it should write `applies_to_component_type = centrifugal_pump` and the KG loader resolves which specific `mbse_entity` nodes are instances of that type at query time.

There are two common exceptions worth noting:

- Some plants maintain component-specific FMEA addenda for high-safety-significance items (unique service conditions, known material susceptibility). These override the class-level RPN with a component-specific one.

- The Maintenance Rule (10 CFR 50.65 in nuclear) assigns functional failure criteria at the functional group level, not the component type level. This maps to safety functions, not failure modes directly.

On NER

Priority	Entity class	Key downstream benefit
1	Temporal expressions [Done]	TSKR construction, latency scoring, causal chain timing
2	Document IDs (CR/WO/ECA/OE/SOP) [Done]	Cross-reference graph edges, evidence bundle precision
3	Alarm IDs [Done]	Alarm signal scoring from narrative text, not just operational_context
4	Process variable tags + values [Done]	Quantitative evidence matching vs. telemetry_summary
5	Procedure / regulatory references	Governance scoring from citations, not just keywords
6	Train / division designators	Topological structural scoring precision

Unit test suite for RCA workflow

test_evidence_scorer.py — `_assess_hit_against_candidate` (the most bug-prone function in the pipeline. Tests each scoring pathway in isolation)

Test	What it pins
<code>test_contradiction_cue_suppressed_when_no_semantic_overlap</code>	The Fix 4 invariant: snippet with zero candidate-term overlap never reaches <code>contradicting</code> role even with cues present
<code>test_contradiction_cue_applied_when_overlap_present</code>	Full +0.45 boost fires when snippet terms match candidate
<code>test_support_cue_adds_to_support_score</code>	<code>caused by / degraded / leak</code> cues raise support
<code>test_candidate_query_type_adds_support_bonus</code>	<code>query_type="candidate"</code> → +0.15 on support
<code>test_contradiction_query_type_adds_contradiction_bonuses</code>	<code>query_type="candidate_contradiction"</code> → +0.15 on contradiction
<code>test_authority_weight_scales_all_scores</code>	mandatory vs guidance vs informational multipliers
<code>test_eca_epistemic_weight_boosts_scores</code>	<code>doc_type=ECA</code> gets 1.2× epistemic multiplier
<code>test_cr_epistemic_weight_discounts_scores</code>	<code>doc_type=CR</code> (preliminary) gets 0.80×
<code>test_role_assignment_contradicting</code>	<code>contradiction ≥ max(support, context)</code> and <code>≥ 0.35</code> → <code>role=contradicting</code>
<code>test_role_assignment_supporting</code>	support dominates → <code>role=supporting</code>
<code>test_role_assignment_contextual_fallback</code>	neither threshold met → <code>role=contextual</code>

test_evidence_dedup.py — `_dedupe_and_rank`

Test	What it pins
test_same_snippet_different_candidates_both_kept	Per-candidate dedup: snippet A linked to cand-1 AND cand-2 both survive
test_higher_score_wins_within_same_candidate_snippet	Two hits for same (snippet, candidate) → keep higher score
test_context_hits_excluded_when_snippet_claimed_by_candidate	Non-candidate hit for an already-claimed snippet_id is dropped
test_context_hits_kept_for_unclaimed_snippets	Non-candidate hit for snippet with no candidate link survives
test_output_sorted_by_score_descending	Final list is score-ordered
test_empty_input_returns_empty	No crash on []

test_evidence_summary.py — build_candidate_evidence_summary

Test	What it pins
test_supporting_count_increments_for_supporting_role	Only support_role=supporting hits count
test_contradicting_count_and_best_score_tracked	best_contradiction_score = max across contradicting hits
test_hits_without_linked_candidate_id_ignored	Unlinked hits don't appear in any summary entry
test_multiple_candidates_aggregated_independently	Two candidates in same bundle produce two separate summaries
test_output_sorted_by_support_minus_half_contradiction	Ordering formula - (best_support - 0.5*best_contradiction)

test_causality_scoring.py — shared scoring helpers (v31 + v32)

Test	What it pins
test_symptom_match_exact_pattern_match_scores_high	gradual_drift observed + gradual_drift expected → 1.0
test_symptom_match_pattern_mismatch_penalizes	step_change observed vs gradual_drift expected → < 0.5
test_symptom_match_no_expected_symptom_types_neutral	FM with no expected_symptom_types → 0.5
test_combine_scores_weighted_sum_arithmetic	5-weight dot product with known inputs
test_combine_scores_clamped_to_unit_interval	weights summing > 1 still produce output in [0,1]
test_candidate_meets_threshold_requires_both_gates	composite OK + evidence NOT OK → fails; both OK → passes

test_causality_engine_generate.py — generate() stage (v31 + v32)

Test	What it pins
test_fm_candidates_and_event_analogs_separated	Fix 2: candidates contains only FM:: IDs; event_analogs contains only EVENT:: IDs

Test	What it pins
test_top_k_applies_only_to_fm_candidates	With top_k=2 and 3 FMs + 2 events, output has ≤ 2 FMs, events unaffected
test_candidates_sorted_by_composite_score_descending	First candidate has highest composite
test_below_threshold_candidates_go_to_filtered_out	FM below both thresholds lands in <code>filtered_out_candidates</code>
test_filtered_out_candidates_are_compact	filtered entries lack full score breakdown (compact form)
test_empty_failure_modes_returns_no_candidates	kg_context with no failure_modes → empty candidates

test_refine_with_evidence.py — v32 refine_with_evidence() only

Test	What it pins
test_high_support_low_contradiction_passes_threshold	refined ≥ 0.35 → <code>meets_evidence_threshold=True</code>
test_high_contradiction_drops_below_threshold	formula $0.30 * \text{prior} + 0.55 * \text{sup} - 0.45 * \text{cntr}$ with adversarial inputs
test_refined_score_updates_composite_and_ranking	After refinement, candidate with better evidence moves up in sort order
test_evidence_gap_flagged_when_no_hits_retrieved	<code>hit_count=0</code> → <code>evidence_gap=True</code> , not filtered as "contradicted"
test_entity_boost_applied_when_entity_matches	NER entity resolved to candidate FM → <code>support_score</code> boosted, refined recomputed
test_zero_evidence_prior_still_passes_with_strong_support	<code>prior=0</code> , <code>support=0.9</code> , <code>contradiction=0</code> → passes threshold

test_synthesizer_validation.py — _validate_card_semantics

Test	What it pins
test_valid_card_produces_no_errors	A well-formed minimal card → []
test_missing_decision_status_flagged	<code>executive_summary.decision_status</code> absent → error
test_evidence_linked_to_unknown_candidate_flagged	evidence item with <code>linked_candidate_id</code> not in <code>primary+alternatives</code> → error
test_evidence_linked_to_primary_accepted	evidence linked to primary → no error
test_evidence_linked_to_alternative_accepted	evidence linked to listed alternative → no error
test_evidence_linked_to_none_accepted	<code>linked_candidate_id=None</code> → no error
test_schema_valid_true_when_zero_errors	<code>len(errors)==0</code> → <code>card.validation_status.schema_valid=True</code>

test_synthesizer_fallback.py — _fallback_card evidence scoping (Fix for A10)

Test	What it pins
test_fallback_strips_linked_id_for_non_card_candidate	evidence linked to retained-but-not-alternative candidate → <code>linked_candidate_id=None</code>

Test	What it pins
<code>test_fallback_preserves_linked_id_for_primary</code>	evidence linked to primary → ID preserved
<code>test_fallback_preserves_linked_id_for_alternatives</code>	evidence linked to top-2 alternatives → IDs preserved
<code>test_fallback_card_passes_validate_card_semantics</code>	full fallback output has zero validation errors

test_query_builder.py — _build_queries

Test	What it pins
<code>test_each_candidate_gets_support_and_contradiction_query</code>	N candidates → 2N candidate-linked query plans
<code>test_candidate_queries_carry_candidate_id</code>	Each query plan has <code>candidate_id</code> set
<code>test_failure_mode_query_added_when_kg_has_fms</code>	KG context with <code>failure_modes</code> → one <code>failure_mode</code> query plan
<code>test_oe_query_scoped_to_oe_doc_ids</code>	OE query plan has <code>doc_ids</code> restricted to OE documents
<code>test_no_top_k_slice_all_candidates_get_queries</code>	Fix 3: 5 candidates → 10+ candidate-linked queries (no [:3] cap)
<code>test_fallback_query_added_when_no_candidates</code>	Empty candidates → one fallback query

Stage-boundary integration tests

test_stage_de_contract.py — causality engine → evidence retriever handoff

Test	What it pins
<code>test_candidates_list_is_list_of_dicts_with_candidate_id</code>	Output of <code>generate()</code> has candidates usable by retriever
<code>test_retriever_produces_summary_for_each_candidate</code>	<code>candidate_evidence_summary</code> has one entry per candidate
<code>test_evidence_score_range_valid</code>	All <code>best_support_score</code> , <code>best_contradiction_score</code> ∈ [0, 1]

test_stage_ef_contract.py — evidence retriever → refine_with_evidence handoff

Test	What it pins
<code>test_summary_lookup_resolves_all_candidate_ids</code>	Every <code>candidate_id</code> in <code>candidates</code> is resolvable in <code>candidate_evidence_summary</code>
<code>test_missing_candidate_in_summary_treated_as_no_data</code>	FM with no evidence entry → <code>evidence_gap=True</code> , not error

Approximate count: ~55 tests across 9 files. Each test is < 30 lines — a minimal fixture dict, one call, one or two asserts.

Priority order to build them (highest ROI first):

- `test_evidence_scorer.py` + `test_evidence_dedup.py` — these guard the two bugs we just fixed
- `test_refine_with_evidence.py` — the v32-specific threshold logic
- `test_causality_engine_generate.py` — FM/analog separation (Fix 2)
- `test_synthesizer_validation.py` + `test_synthesizer_fallback.py` — A10 guard
- `test_causality_scoring.py` — scoring arithmetic
- `test_query_builder.py` + stage contracts — retrieval contract

